



CMPT 295

Unit – Code Optimization

Lecture 35 – Examples of Code Optimization

Last Lecture

Microprocessor machine code instruction execution:

3. Pipelined execution of machine code instructions

➤ Limitations on pipelined execution

...

3. Hazard: 1) **Structural**, 2) **Data** and 3) **Control**

3. **Control hazard** created when microprocessor needs to guess whether to branch/jump or not. The hazard is that the microprocessor may get it wrong, i.e., it may mis-predict. This causes instructions to be squashed, negatively affecting the throughput

➤ Solution:

➤ Simple branch prediction strategies: **always jump** or **never jump**

Today's Menu

- Code optimization
 - Examples of code optimization
 - Done by compilers
 - Done by software developers
 - More optimization strategies - Based on microprocessor architecture
 - Loop unrolling
 - Instruction parallelism
 - Re-association
 - Accumulators

Goal for this unit

- So far, we have learnt
 - What an instruction set architecture (ISA) is
 - We have created a few simple ISA's
 - We have also learnt quite a lot about the **x86-64** ISA and the **MIPS** ISA
 - We have some understanding of how a simple microprocessor executes machine code instructions
 - Design of the datapath of microprocessor
 - Various models of **machine code instruction execution**
- Let's now put all this together and investigate how this can help us make our programs more time efficient (i.e., run faster)
 - By exploring different **program optimization strategies**

Software developer's objectives

- However, let's keep in mind that ...
- Our objective as software developers is to write programs that ...
 - Solve the given problems
 - Are maintainable
 - Are robust
 - And are optimized ... **only if *faster performance*** is required
- Why don't we always optimize our programs?
- Because optimizing programs is ***time consuming*** and ***involves trade-offs***

Capabilities of optimizing compilers

- Employ sophisticated forms of analysis and optimization
 - On our *target machine*, **gcc**: -Og, -O1, -O2
 - Examples:
 1. *Reduction in strength*
 2. *Removing unnecessary function calls*
 3. *Removing unnecessary arguments to function call*
-> argument elision
 4. *Code motion*
 5. *Inlining*

1. *Reduction in Strength* optimization

1. $y = x / 2.0 \Rightarrow y = x * 0.5$

- Division more expensive than multiplication
- If compiler can tell that denominator is a constant, it may use multiplication instead

2. $x = \text{pow}(y, 2.0) \Rightarrow x = y * y$

- Raising a number to a power is more expensive than multiplying (function call also more expensive than multiplying)
- If the compiler can tell that the power is a small number, it may use multiplication instead

2. Removing unnecessary function calls

```
int proc(long a1, int a2, int a3, long a4, char a5, char a6, char a7, long *alp)
{
    return a1+a2+a3+a4+a5+a6+a7; // 28
}

long call_proc()
{
    long x1 = 1;
    int x2 = 2;
    int x3 = 3;
    long x4 = 4;
    char x5 = 5;
    char x6 = 6;
    char x7 = 7;

    long result = proc(x1, x2, x3, x4, x5, x6, x7, &x1);
    return result;
}

void main() {
    call_proc();

    return;
}
```


No Optimization

```
.file "main.c"
.text
.globl proc
.type proc, @function
proc:
.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
movq %rdi, -8(%rbp)
movl %esi, -12(%rbp)
movl %edx, -16(%rbp)
movq %rcx, -24(%rbp)
movl %r8d, %edx
movl %r9d, %ecx
movl 16(%rbp), %eax
movb %dl, -28(%rbp)
movl %ecx, %edx
movb %dl, -32(%rbp)
movb %al, -36(%rbp)
movq -8(%rbp), %rax
movl %eax, %edx
movl -12(%rbp), %eax
addl %eax, %edx
movl -16(%rbp), %eax
addl %eax, %edx
movq -24(%rbp), %rax
addl %eax, %edx
movsbl -28(%rbp), %eax
addl %eax, %edx
movsbl -32(%rbp), %eax
addl %eax, %edx
movsbl -36(%rbp), %eax
addl %edx, %eax
popq %rbp
```

```
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE0:
.size proc, .-proc
.globl call_proc
.type call_proc, @function
call_proc:
.LFB1:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
subq $48, %rsp
movq %fs:40, %rax
movq %rax, -8(%rbp)
xorl %eax, %eax
movq $1, -32(%rbp)
movl $2, -40(%rbp)
movl $3, -36(%rbp)
movq $4, -24(%rbp)
movb $5, -43(%rbp)
movb $6, -42(%rbp)
movb $7, -41(%rbp)
movsbl -41(%rbp), %edi
movsbl -42(%rbp), %r9d
movsbl -43(%rbp), %r10d
movq -32(%rbp), %rax
movq -24(%rbp), %rcx
movl -36(%rbp), %edx
movl -40(%rbp), %esi
leaq -32(%rbp), %r8
pushq %r8
pushq %rdi
movl %r10d, %r8d
```

```
movq %rax, %rdi
call proc
addq $16, %rsp
cltq
movq %rax, -16(%rbp)
movq -16(%rbp), %rax
movq -8(%rbp), %rdx
xorq %fs:40, %rdx
je .L5
call __stack_chk_fail@PLT
.L5:
leave
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE1:
.size call_proc, .-call_proc
.globl main
.type main, @function
main:
.LFB2:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
movl $0, %eax
call call_proc
nop
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE2:
.size main, .-main
.ident "GCC: (Ubuntu 7.5.0-3ubuntu1~18.04) 7.5.0"
.section .note.GNU-stack,"",@progbits
```

-Og -> Basic set of optimizations

Produces
assembly code
that mirrors
the C code
so it can be
debugged
using the
debugger

```
.file "main.c"
.text
.globl proc
.type proc, @function
proc:
.LFB0:
.cfi_startproc
addl    %edi, %esi
addl    %esi, %edx
addl    %edx, %ecx
movsbl  %r8b, %r8d
addl    %r8d, %ecx
movsbl  %r9b, %r9d
addl    %r9d, %ecx
movsbl  8(%rsp), %eax
addl    %ecx, %eax
ret
.cfi_endproc
.LFE0:
.size   proc, .-proc
.globl  call_proc
.type   call_proc, @function
call_proc:
.LFB1:
.cfi_startproc
subq    $24, %rsp
.cfi_def_cfa_offset 32
movq    %fs:40, %rax
movq    %rax, 8(%rsp)
xorl    %eax, %eax
movq    %rsp, %rax
pushq   %rax
.cfi_def_cfa_offset 40
pushq   $7
.cfi_def_cfa_offset 48
movl    $6, %r9d
```

```
movl    $5, %r8d
movl    $4, %ecx
movl    $3, %edx
movl    $2, %esi
movl    $1, %edi
call    proc
addq    $16, %rsp
.cfi_def_cfa_offset 32
movq    8(%rsp), %rdx
xorq    %fs:40, %rdx
jne     .L5
cltq
addq    $24, %rsp
.cfi_remember_state
.cfi_def_cfa_offset 8
ret
.L5:
.cfi_restore_state
call    __stack_chk_fail@PLT
.cfi_endproc
.LFE1:
.size   call_proc, .-call_proc
.globl  main
.type   main, @function
main:
.LFB2:
.cfi_startproc

...
```

-01-> 2. Removing unnecessary function calls

Compiler has removed the call to `proc (...)` and replaced it with the return value of `proc (...)`, i.e., 28!

```
int proc(long a1, int a2, int a3, long a4, char a5, char a6, char a7, long *alp)
{
    return a1+a2+a3+a4+a5+a6+a7; // 28
}

long call_proc()
{
    long x1 = 1;
    int x2 = 2;
    int x3 = 3;
    long x4 = 4;
    char x5 = 5;
    char x6 = 6;
    char x7 = 7;

    long result = proc(x1, x2, x3, x4, x5, x6, x7, &x1);
    return result;
}
```

```
.file "main.c"
.text
.globl proc
.type proc, @function
proc:
.LFB0:
    .cfi_startproc
    movsbl %r9b, %r9d
    movsbl 8(%rsp), %eax
    addl %eax, %r9d
    movsbl %r8b, %r8d
    addl %r8d, %r9d
    leal (%r9,%rcx), %eax
    addl %edx, %eax
    addl %esi, %eax
    addl %edi, %eax
    ret
    .cfi_endproc
.LFE0:
    .size proc, .-proc
    .globl call_proc
    .type call_proc, @function
call_proc:
.LFB1:
    .cfi_startproc
    movl $28, %eax
    ret
    .cfi_endproc
.LFE1:
    .size call_proc, .-call_proc
    .globl main
    .type main, @function
main:
.LFB2:
    .cfi_startproc
    ...
```

3. Argument Elision

- Actual code:

```
int f(x) {  
    return 1 + g(x);  
}
```

- Compiler transforms it into machine code equivalent to:

```
int f(x) {  
    return 1 + g();  
}
```

gcc realizes that there is no need to set up the first argument to `g(x)` within `f(x)`, i.e., move `x` in the register **%edi** since `x` is already in **%edi**

- Here is a section of the assembly code equivalence of `f(x)`:

```
subq    $8, %rsp  
call    _Z1gi@PLT  
addq    $8, %rsp  
addl    $1, %eax  
ret
```

4. Compiler-Generated Code Motion (-O1)

```
void setRow(char *A, char *B, int i, int N)
{
    int j;
    for (j = 0; j < N; j++)
        A[N*i+j] = B[j];
}
```

```
int j;
int Ni = N*i;
long size = B+N;
for (j = 0; B < size; B++){
    *(A+Ni) = *B;
    Ni++;
}
```

```
setRow:
    testl    %ecx, %ecx           # Test N
    jle      done                # If 0, done
    imull    %ecx, %edx           # Ni = N*i
    movq     %rsi, %rax
    leal     -1(%rcx), %ecx       # N-1
    leaq     1(%rsi,%rcx), %r8    # size = B+N
loop:
    movzbl   (%rax), %esi        # t = M[B]
    movslq   %edx, %rcx
    movb     %sil, (%rdi, %rcx)  # M[A+Ni] = t
    addl     $1, %edx            # Ni++
    addl     $1, %rax            # B++
    cmpq     %r8, %rax           # if B != B+N ...
    jne      loop               # ... loop
done:
    ret
```

5. *Inlining* - Demo

- Posted demo which illustrates **gcc** optimizing function call by inline substitution
 1. [func1_2.c](#) and [func1_2_O2.s](#) - assembly code with -O2 illustrate a situation in which **gcc** is blocked from inlining
 - ***optimization blocker***
 2. [func_all.c](#), [func_all_O2.s](#) - assembly code with -O2 illustrate a situation in which **gcc** is able to optimize function call by inlining

Limitations of optimizing compilers

- Ideally, compilers able to take whatever code we write and generate the most efficient possible machine code program
 - Realistically, compilers are stopped by **optimization blockers**:
 - **Function calls**
 - Compilers may not be able to determine whether a function is free of side effect – compilers may leave functions calls intact as opposed to inlining
 - **Memory aliasing**
 - Compilers may not be able to determine whether or not two pointers are aliased - pointing to same memory location - compilers may not optimize the use of the pointers
 - Why?
 - Compilers must produce a machine code version of our program with same behavior for all possible test cases
 - **Solution**: Help compilers by rewriting our code

Summary

- If fast performance is required, s/w developers can optimize programs (make them more time efficient) by exploring various program optimization strategies
 - Compiler-generated **optimization strategies**:
 1. Reduction in Strength
 2. Removing unnecessary function call
 3. Removing unnecessary function arguments argument elision
 4. Code Motion
 5. *Inlining* (inline substitution)
 - When compiler can perform optimization (not blocked)
 - Compiler optimization blockers:
 1. Function calls
 2. Memory aliasing

Next Lectures

- ▀ Code optimization
 - ▀ Examples of code optimization
 - ▀ Done by compilers
 - ▀ Done by software developers
 - ▀ More optimization strategies - Based on microprocessor architecture
 - ▀ Loop unrolling
 - ▀ Instruction parallelism
 - ▀ Re-association
 - ▀ Accumulators